```
{string:{trim,pad},array:{remove},stats:{chebyshev}} = require
'./utils/index'
```

This creates `trim`, `pad`, `remove`, and `chebyshev` variables in the local module that reference the properties of the same name exported from the `string`, `array`, and `stats` modules via the `index` module. To make things a bit easier, the Node.js module system knows to look for an index file implicitly when a directory name is supplied:

```
{string:{trim,pad},array:{remove},stats:{chebyshev}} = require
'./utils'
```

The module system has allowed your program to still run correctly after it has been broken into individual files. Remember, though, that your CoffeeScript program doesn't actually run as CoffeeScript. It compiles to JavaScript and runs as JavaScript. Once your program gets bigger, it becomes impractical to individually compile all of the source files to JavaScript and then run it. Although the command-line CoffeeScript compiler is capable of compiling multiple files, your build eventually becomes too complicated to do it all manually each time, especially if things need to be done in a particular order. You need an automated build.

How about packages?

You'll notice that you've been installing Node.js programs (and modules) by using npm. This is because npm is how you install external modules, manage those modules, and also package your own modules so that they can be installed using npm. Although npm is an important part of the Node.js ecosystem, a deep exploration of it is beyond the scope of this book. The official npm documentation is available via https://npmjs.org/ and is a good place to start exploring npm.

12.2. Build automation with Cake